

Computer Vision HW Assignment 2 Solution

Yarden Carmel, 208520288

Yoav Shaked, 315152629

Part A

1. To compute the different ssdd values we look at the images:

Left				
1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

Right				
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4

For (1,2,2):

$$\begin{aligned}ssdd(1,2,2) &= ssd\left(\begin{bmatrix} 2 & 3 & 4 \\ 7 & 8 & 9 \\ 12 & 13 & 14 \end{bmatrix}, \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}\right) = \begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}^2 \\&= 1 + 4 + 9 + 25 + 36 + 49 + 81 + 100 + 121 = 426\end{aligned}$$

For (1,2,3):

$$\begin{aligned}ssdd(1,2,3) &= ssd\left(\begin{bmatrix} 2 & 3 & 4 \\ 7 & 8 & 9 \\ 12 & 13 & 14 \end{bmatrix}, \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}\right) = \begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}^2 \\&= 1 + 4 + 9 + 25 + 36 + 49 + 81 + 100 + 121 = 426\end{aligned}$$

For (2,3,0):

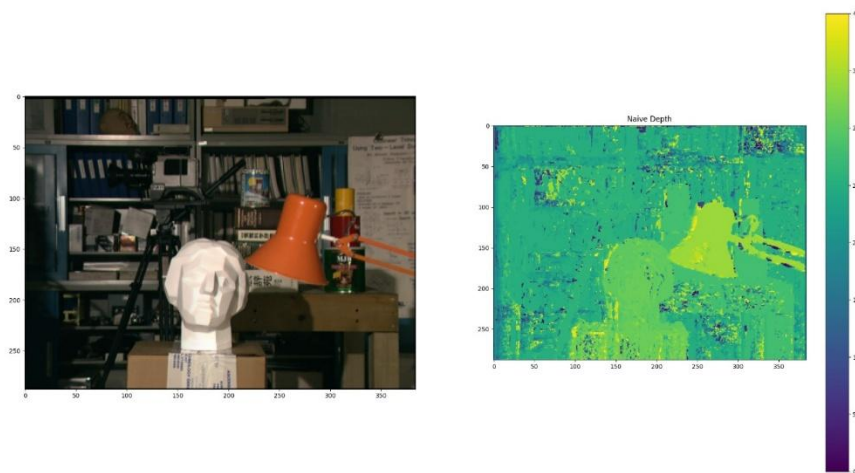
$$\begin{aligned}ssdd(2,3,0) &= ssd\left(\begin{bmatrix} 8 & 9 & 10 \\ 13 & 14 & 15 \\ 18 & 19 & 20 \end{bmatrix}, \begin{bmatrix} 2 & 2 & 2 \\ 3 & 3 & 3 \\ 4 & 4 & 4 \end{bmatrix}\right) = \begin{bmatrix} 6 & 7 & 8 \\ 10 & 11 & 12 \\ 14 & 15 & 16 \end{bmatrix}^2 \\&= 36 + 49 + 64 + 100 + 121 + 144 + 196 + 225 + 256 = 1191\end{aligned}$$

For (2,3,1):

$$\begin{aligned}ssdd(2,3,1) &= ssd\left(\begin{bmatrix} 8 & 9 & 10 \\ 13 & 14 & 15 \\ 18 & 19 & 20 \end{bmatrix}, \begin{bmatrix} 2 & 2 & 2 \\ 3 & 3 & 3 \\ 4 & 4 & 4 \end{bmatrix}\right) = \begin{bmatrix} 6 & 7 & 8 \\ 10 & 11 & 12 \\ 14 & 15 & 16 \end{bmatrix}^2 \\&= 36 + 49 + 64 + 100 + 121 + 144 + 196 + 225 + 256 = 1191\end{aligned}$$

Part B

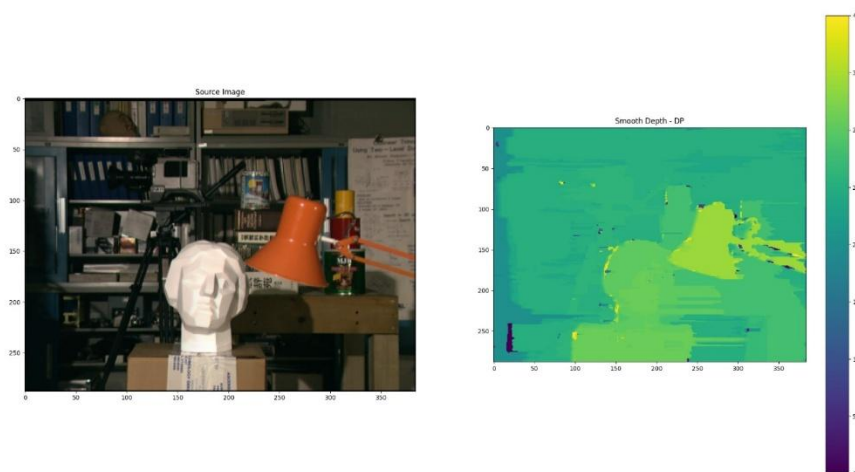
4. The result:



In the result we can see an inconsistent labeling of neighboring pixels. This happens because choosing the disparity value naively from the SSDD matrix makes the algorithm favor disparity which bring each pixel in the first image closer to a similar pixel in a similar neighborhood in the second image (for which the SSD is lower), causing an inconsistency which contradicts real-world logic. This is an example of the local method's lack of global context when labeling (although it does take neighbors into account, it is not enough). We note that this happens mostly in edges and contours of background objects due to how similar their intensities are, causing great confusion for the algorithm.

Part C

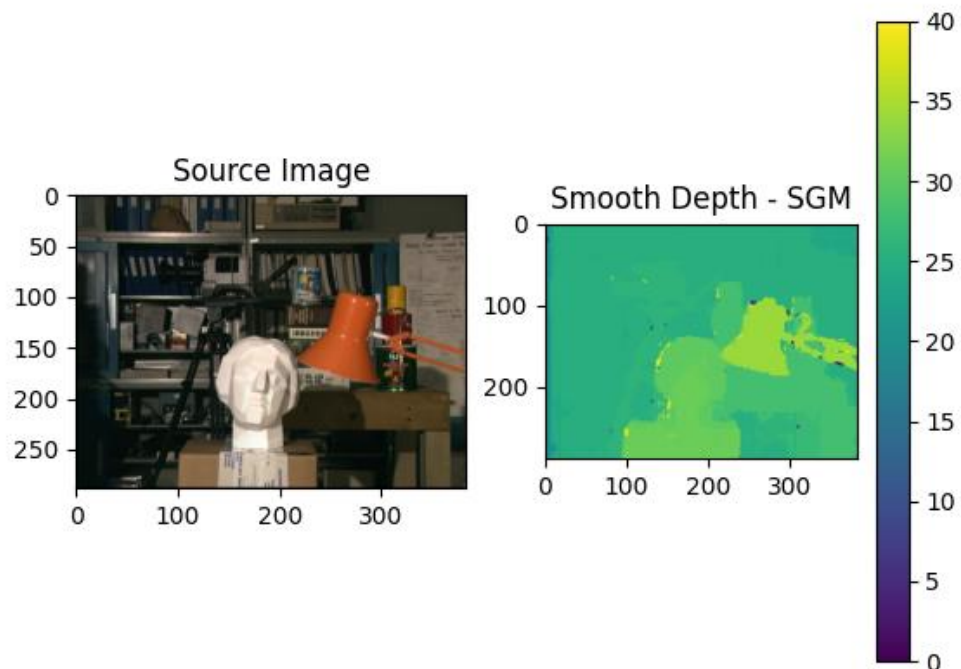
7. The result for smooth depth:



It is easy to see that there are far less "noisy" labels in the smooth depth in comparison to the naïve depth. We also note that there seem to be several artifacts where the depth is "smeared" along a line, this comes from the fact that we applied the smoothing over the lines using the semi-global method taught in class. This also makes obvious the streaking problem that the semi-global method is known for – namely, the inconsistency in depth between neighboring scan lines which isn't evident in the naïve depth.

Part D

11. The result:



We notice that the SGM depth map is much better than the naïve one. It is smoother and has fewer artifacts, as expected from the Semi-Global method. This method still has artifacts near the edges of certain objects which can be attributed to the SGM's reliability on the penalty score. It cannot be guaranteed that by increasing or decreasing the penalty we would get a better result, but it could help smooth the image further.

Part E

Our images:

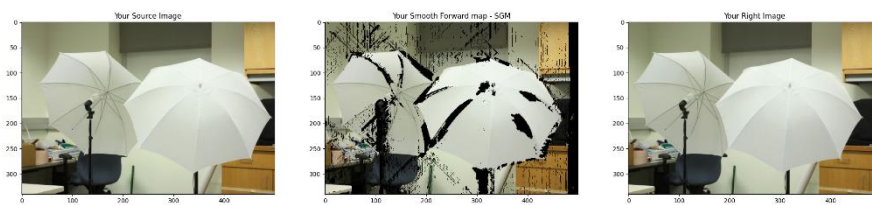
My_image_left:



My_image_right:



The different results:



Even here we see the SGM being less effective near edges in areas of relatively small change in intensity, which can be attributed to the choice of the penalty score.